

Gradient Descent — the whole picture

finding the parameter that makes a model's error as small as possible

1 · THE INTUITION

A hiker in fog wants the valley floor.
Can't see far, but can feel the slope.
Step downhill, feel again, repeat.

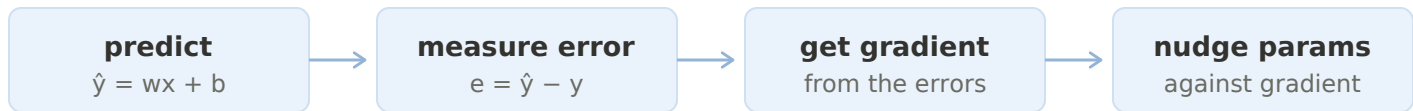
loss = the landscape · gradient = the slope
learning rate = how big each step is

2 · THE UPDATE RULE

$$\theta = \theta - \alpha \cdot \nabla L(\theta)$$

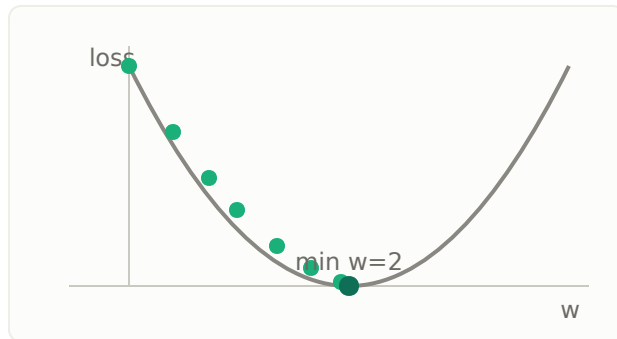
θ the parameter you're tuning
 α learning rate (step size)
 ∇L gradient — points uphill, so subtract it

3 · THE TRAINING LOOP (repeat until loss stops falling)



4 · SIMPLEST EXAMPLE

One point ($x=1, y=2$), model $\hat{y} = wx$.
Loss $L = (w - 2)^2$. Start $w=0, \alpha=0.1$.



dot rolls downhill; w climbs $0 \rightarrow 2$

5 · WHERE THE GRADIENT COMES FROM

Differentiate $L = (w - 2)^2$ via chain rule:

outer: $d/du (u^2) = 2u$

inner: $d/dw (w - 2) = 1$

$$\rightarrow \partial L / \partial w = 2(w - 2) \cdot 1 = 2 \cdot e$$

why $(u)^2$ has slope $2u$:

grow a square's side by a hair h —
two strips ($2 \cdot u \cdot h$) appear, corner h^2 is
negligible. Divide by $h \rightarrow 2u$.

general (2 knobs): $\partial L / \partial w = 2 \cdot e \cdot x, \partial L / \partial b = 2 \cdot e$

THE ONE-LINE TAKEAWAY

Data makes the errors $\rightarrow$ errors make the gradient $\rightarrow$ gradient nudges the parameters $\rightarrow$
repeat thousands of times. That loop, scaled to millions of parameters, IS training a model.

learning rate matters most: too small = crawls · too big = overshoots and diverges · just right = smooth descent

Two spaces to keep separate: data space (x vs y — the line $y=2x$ you're learning) vs parameter space (w vs loss — the parabola y).
The bottom of the parabola is the value of w that produces the correct line.